Brian Benedicks **Duke AIPI 561**

Week 7 - Optimization of Techcorp Agent

AIPI 561: Operationalizing AI

This report covers cost_optimization_starter.py, which implements CostAnalyzer, OptimizationStrategy, and FeedbackLoop. The script was run with python cost_optimization_starter.py and completed with no errors. The output below is the evidence for cost breakdown, spike detection, optimization strategies, and the feedback loop.

CostAnalyzer records every query along with its retrieval, LLM, tool, and error costs, then totals them by component in get_cost_breakdown().

To find unusually expensive queries, identify_cost_spikes() calculates the mean and standard deviation of total_cost across all recorded queries and flags anything more than two standard deviations above the mean. In the test run, ten normal-cost queries and one deliberately expensive query were recorded, and the spike detector correctly isolated only the expensive one.

OptimizationStrategy applies four cost-saving techniques. apply_caching() stores past query/response pairs in an LRU OrderedDict (with a max size, so it does not grow without bound) and returns a cache hit when the same query repeats. select_model_by_complexity() checks for keywords like analyze, compare, explain, and design, and routes those queries to the stronger gemini-2.5-pro while routing simpler lookups to the cheaper gemini-1.5-flash. optimize_retrieval_count() reduces the number of documents retrieved per query (15 down to 3 in the test run), and enable_response_compression() shortens long responses down to their first couple of sentences. get_optimization_impact() then reports a savings percentage and a per-strategy breakdown calculated from what these methods actually did during the run, not a fixed estimate.

FeedbackLoop collects corrections submitted by users and decides whether to accept them.

Rejected corrections are still stored along with the reason for rejection, so there is a full audit trail. save_corrections() and load_corrections() write the corrections list to data/corrections.json and read it back, so feedback is not lost when the script restarts.

Below is the full console output from the __main__ test block.

```
======================================================================
TASK 1: Testing CostAnalyzer
======================================================================
INFO:__main__:Recorded query 'What is the travel policy?'
total_cost=$0.005000
INFO:__main__:Recorded query 'Who can approve expenses over $5000?'
total_cost=$0.007500
INFO:__main__:Recorded query 'What is the NYC per diem rate?'
total_cost=$0.004000
INFO:__main__:Recorded query 'What is the compensation policy?'
total_cost=$0.005000
INFO:__main__:Recorded query 'What is Alice's role?' total_cost=$0.004300
```

```
INFO:__main__:Recorded query 'Look up employee John Smith.'
total_cost=$0.006700
INFO:__main__:Recorded query 'What is the expense approval limit for an
engineer' total_cost=$0.004800
INFO:__main__:Recorded query 'What is the total expenses for the Alpha
project?' total_cost=$0.006500
INFO:__main__:Recorded query 'What medical benefits are available to
employees?' total_cost=$0.004500
INFO:__main__:Recorded query 'What is the per diem rate for Chicago?'
total_cost=$0.004300
INFO:__main__:Recorded query 'Analyze and compare expense trends across all
depa' total_cost=$1.000000

Cost breakdown across all recorded queries:
  retrieval_total: 0.4605
  llm_total: 0.4584
  tool_total: 0.0837
  error_total: 0.05
  total_daily: 1.0526
  query_count: 11

Cost spikes detected: 1
  SPIKE: 'Analyze and compare expense trends across all departments fo'
cost=$1.0000 (threshold=$0.6955, mean=$0.0957, stdev=$0.2999)


======================================================================
TASK 2: Testing OptimizationStrategy
======================================================================

-- Caching --
  First call  -> is_cached_hit=False, response='Answer A'
  Second call -> is_cached_hit=True, response='Answer A'
  [OK] Cache returns identical response on repeat query.

-- Model selection by complexity --
  Simple query  -> 'What is the per diem rate for NYC?' routed to: gemini-
1.5-flash
  Complex query -> 'Analyze and compare expense trends across departments'
routed to: gemini-2.5-pro
  [OK] Simple queries use the cheaper model, complex queries use the stronger
one.

-- Retrieval count optimization --
  15 documents requested -> optimized to 3 documents

-- Response compression --
  Original (228 chars): The travel policy allows business class for flights
over 8 hours. Economy class is standard for shorter flights. All bookings
must be made through the corporate travel portal. Receipts must be submitted
within 30 days of travel.
  Compressed (112 chars): The travel policy allows business class for flights
over 8 hours. Economy class is standard for shorter flights.

-- Optimization impact summary --
```

```
{
  "total_savings_pct": 56.67,
  "strategies_applied": [
    "caching",
    "model_selection",
    "retrieval_reduction",
    "response_compression"
  ],
  "breakdown": {
    "caching": {
      "cache_hit_rate_pct": 50.0,
      "calls_served_from_cache": 1,
      "total_cache_calls": 2
    },
    "model_selection": {
      "pct_routed_to_cheaper_model": 50.0,
      "estimated_savings_pct": 40.0,
      "selections": {
        "gemini-1.5-flash": 1,
        "gemini-2.5-pro": 1
      }
    },
    "retrieval_reduction": {
      "avg_doc_count_reduction_pct": 80.0,
      "reductions_applied": 1
    },
    "response_compression": {
      "responses_compressed": 1
    }
  }
}
```


```
======================================================================
TASK 3: Testing FeedbackLoop
======================================================================
INFO:__main__:Correction submitted by role=manager accepted=True
  Role=manager    accepted=True  reason=Correction accepted: sufficient
authority and added detail.
INFO:__main__:Correction submitted by role=engineer accepted=False
  Role=engineer   accepted=False reason=Role 'engineer' (authority level 1)
lacks sufficient authority; manager-level (3) or above required.
INFO:__main__:Correction submitted by role=executive accepted=False
  Role=executive  accepted=False reason=Corrected answer is not more detailed
than the original (16 chars vs 111 chars).
INFO:__main__:Correction submitted by role=hr accepted=False
  Role=hr         accepted=False reason=Role 'hr' (authority level 2) lacks
sufficient authority; manager-level (3) or above required.
INFO:__main__:Correction submitted by role=executive accepted=True
  Role=executive  accepted=True  reason=Correction accepted: sufficient
authority and added detail.

-- Feedback metrics --
```


```
{
  "total_corrections": 5,
  "validation_rate": 0.4,
```

```
  "avg_correction_length": 68.8,
  "top_error_patterns": [
    { "pattern": "There is no specific", "count": 1 },
    { "pattern": "SSNs are 9 digits.", "count": 1 },
    { "pattern": "Compensation is reviewed annually", "count": 1 },
    { "pattern": "Some benefits exist.", "count": 1 },
    { "pattern": "It varies.", "count": 1 }
  ]
}

-- Persisting corrections to disk --
INFO:__main__:Saved 5 corrections to data/corrections.json
INFO:__main__:Loaded 5 corrections from data/corrections.json
  Reloaded 5 corrections from data/corrections.json
  [OK] Corrections persist and reload correctly.

======================================================================
ALL TESTS COMPLETED
======================================================================
```

data/corrections.json was checked after the run and contains all 5 correction entries with the correct accepted/reason/timestamp values, matching the output above.

All three classes behaved as expected with no errors, and the test block's assertions all passed.

This covers the cost breakdown, spike detection, optimization strategies, and feedback loop required for this assignment.